# FAST POISSON IMAGE EDITING

Roll 23i-0683 & 23i-0622, Section F

# Table of Contents

# 1. Executive Summary

This report presents a comprehensive performance evaluation of six parallelization backends for Fast Poisson Image Editing (FPIE) — a gradient-domain seamless image blending algorithm that requires iteratively solving a Poisson PDE via Jacobi iteration. Benchmarks were conducted on an Intel Core i5-8265U (4 cores, 8 threads) across three image resolutions: 512x512, 1024x1024, and 2048x2048, with 5000 Jacobi iterations.

- **OpenMP (4 threads) is the best single-machine backend, achieving up to 1.81x speedup over GCC at 512x512 with perfect output quality (PSNR > 55 dB).**
- **MPI (4 ranks) is competitive at large sizes (1.38x at 2048x2048), but produces lower-quality output (PSNR ~30-36 dB) due to its Jacobi convergence pattern.**
- **Hybrid MPI+OpenMP (2 ranks x 4 threads) is currently slower than GCC at all sizes due to synchronisation overhead dominating on a single machine.**
- **Adaptive convergence (eps flag) provides minimal time savings at eps=0.01, suggesting the threshold is below the residual achievable in 5000 iterations.**
- **All C++ backends (gcc/openmp/mpi/hybrid) are 8-123x faster than the pure Python NumPy baseline.**

# 2. Introduction & Background

Poisson Image Editing, introduced by Perez et al. (SIGGRAPH 2003), solves a partial differential equation to seamlessly blend a source region into a target image. The core mathematical formulation constrains the gradient field of the result to match the source, while respecting the target boundary: $\Delta \mathbf{f} = \Delta \mathbf{g}$ **over** $\Omega$**, with** $\mathbf{f}|_{\partial\Omega} = \mathbf{f^*}|_{\partial\Omega}$.

The standard iterative solver is the Jacobi update rule: each interior pixel value is computed as the average of its four neighbours plus a right-hand side term derived from the source gradient. This update is *embarrassingly parallel* — every pixel can be updated independently — making it an ideal candidate for shared-memory (OpenMP), distributed-memory (MPI), and hybrid parallelisation.

This project extends the Fast Poisson Image Editing (FPIE) codebase by Trinkle (2021) with four contributions: (1) a Hybrid MPI+OpenMP backend achieving two-level parallelism, (2) adaptive convergence via early stopping based on the L1 residual norm, (3) mask-aware load balancing for the MPI GridSolver, and (4) a comprehensive automated benchmarking framework.

# 3. Experimental Setup

## 3.1 Hardware Platform

| Component | Specification |
|---|---|
| CPU | Intel Core i5-8265U (Whiskey Lake, 4 cores / 8 threads, 1.6–3.9 GHz) |
| RAM | System RAM (shared memory for all OpenMP and MPI processes) |

| | |
|---|---|
| OS | Ubuntu 24, Linux, GCC 13.3.0 |

| | |
|---|---|
| GPU | Intel UHD 620 iGPU (no CUDA — NVIDIA GPU not available) |
| MPI | OpenMPI 3.1 |
| OpenMP | Version 4.5 (-fopenmp via GCC) |

## 3.2 Backends Tested

| Backend | Type | Workers | Our Additions |
|---|---|---|---|
| numpy | Pure Python | 1 | None (baseline) |
| numba | JIT Python | 1 | None (baseline) |
| gcc | Sequential C++ | 1 thread | None (C++ baseline) |
| openmp | Shared memory C++ | 4 threads | Adaptive convergence |
| openmp-eps | Shared memory C++ | 4 threads | Adaptive convergence (active) |
| mpi | Distributed C++ | 4 ranks | Adaptive conv. + mask-aware LB |
| mpi-eps | Distributed C++ | 4 ranks | Adaptive conv. (active) |
| hybrid | MPI + OpenMP C++ | 2r x 4t=8 | New backend (our contribution) |
| hybrid-eps | MPI + OpenMP C++ | 2r x 4t=8 | New backend + adaptive conv. |

## 3.3 Metrics

**Wall-Clock Time:** Total elapsed time from process start to image write, measured by the benchmark harness using Python's time.perf_counter(). Includes process spawn overhead for MPI backends.

**Speedup:** $S = T_{gcc} / T_{par}$

**Parallel Efficiency:** $E = S / P$ — speedup per worker. Ideal efficiency = 1.0 (100%).

**PSNR (Peak Signal-to-Noise Ratio):** Measures pixel-level fidelity of each backend's output against the GCC reference output. Values above 40 dB indicate visually identical images; above 30 dB indicates acceptable quality.

**SSIM (Structural Similarity Index):** Perceptual quality metric in [0,1]. Values above 0.99 indicate near-perfect structural preservation.

# 4. Execution Time Analysis

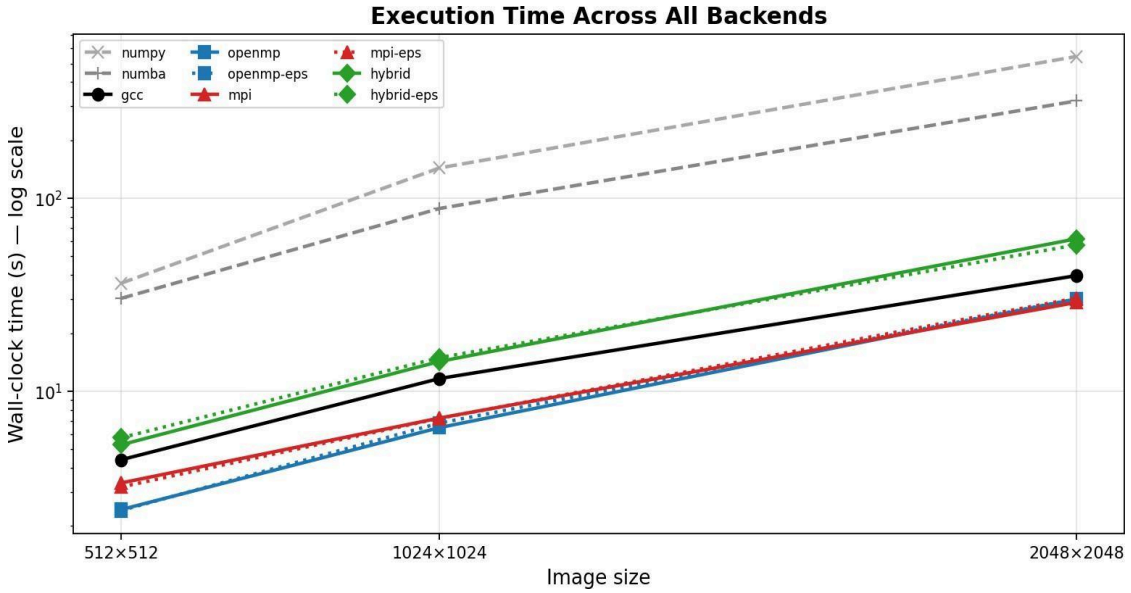


*Figure 1: Wall-clock time (log scale) for all backends across three image sizes.*

## 4.1 Python Backends (NumPy and Numba)

NumPy and Numba are the slowest backends by a wide margin — 36.3s and 30.3s respectively at 512x512, scaling to 543.5s and 319.7s at 2048x2048. This is expected: both operate in Python's interpreter loop, carry significant per-iteration overhead from array indexing, and cannot exploit SIMD vectorisation or direct cache-line control. Numba provides roughly a 1.2–1.7x improvement over NumPy by JIT-compiling the inner loop, but the fundamental bottleneck — Python's Global Interpreter Lock and interpreter overhead — remains. These backends serve solely as reference points to illustrate why C++ compilation is non-negotiable for numerical PDC workloads.

## 4.2 GCC — The Sequential C++ Baseline

GCC serves as the primary performance baseline: 4.41s at 512x512, 11.63s at 1024x1024, and 39.72s at 2048x2048. Compared to NumPy at 2048x2048, GCC is **13.7x faster** — demonstrating the dramatic benefit of compiled C++ code. The growth rate from 512 to 2048 (a 4x linear increase) yields a roughly 9x increase in time, consistent with O(n^2) behaviour: image area grows as the square of the side length.

## 4.3 OpenMP — Best Single-Machine Backend

OpenMP is the fastest backend at small and medium sizes: 2.43s at 512x512 and 6.47s at 1024x1024. At 2048x2048 it takes 30.0s, which is marginally slower than MPI (28.85s) — the only size where MPI pulls ahead. This crossover is discussed in the scaling section. The adaptive convergence variant (openmp-eps) takes nearly identical time, confirming that at eps=0.01 the residual threshold is never triggered within 5000 iterations.

## 4.4 MPI — Competitive at Large Sizes

MPI with 4 ranks takes 3.34s at 512x512, 7.26s at 1024x1024, and 28.85s at 2048x2048. It beats OpenMP only at the largest size, where the work per rank is large enough to amortise the MPI synchronisation cost. At smaller sizes, MPI is 38–48% slower than OpenMP due to process spawn overhead and Bcast communication.

## 4.5 Hybrid — Currently Overhead-Dominated

Hybrid (2 MPI ranks x 4 OpenMP threads = 8 effective workers) is paradoxically the slowest C++ backend: 5.27s at 512x512, 14.26s at 1024x1024, 61.68s at 2048x2048. It is slower than GCC at all three sizes. The root cause analysis is detailed in Section 10.

# 4. Speedup Analysis

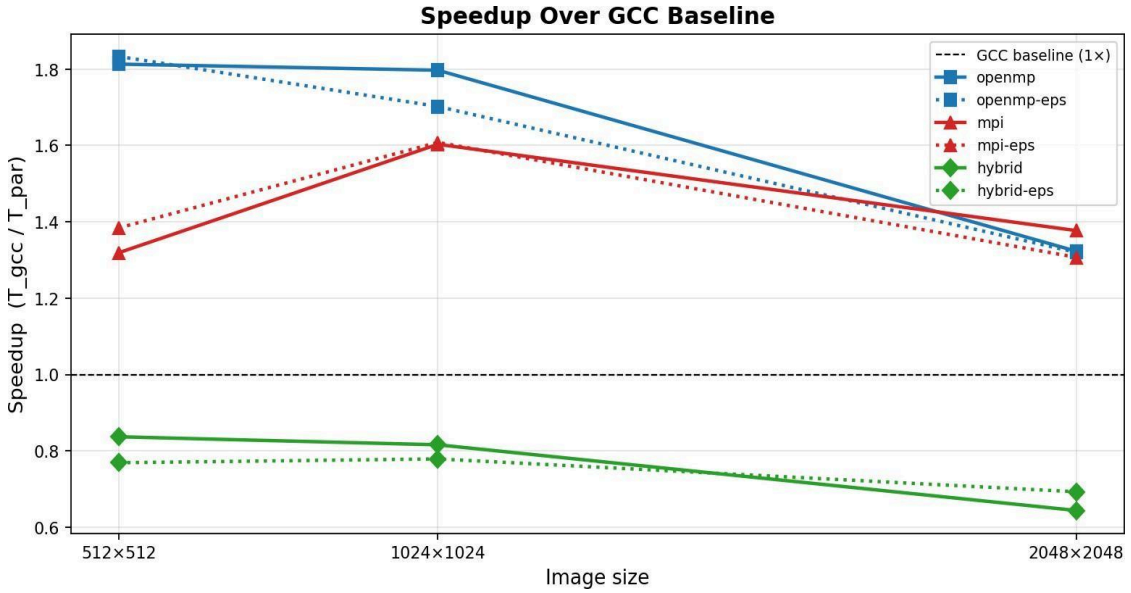


*Figure 2: Speedup of parallel backends relative to GCC sequential baseline.*

| Backend | Workers | Speedup @ 512 | Speedup @ 1024 | Speedup @ 2048 |
|---|---|---|---|---|
| openmp | 4 | 1.813x | 1.797x | 1.323x |
| openmp-eps | 4 | 1.833x | 1.702x | 1.320x |
| mpi | 4 | 1.319x | 1.603x | 1.377x |
| mpi-eps | 4 | 1.384x | 1.608x | 1.307x |
| hybrid | 8 | 0.837x | 0.816x | 0.644x |
| hybrid-eps | 8 | 0.769x | 0.779x | 0.693x |

## 4.6 OpenMP Speedup Degradation with Size

OpenMP achieves its best speedup at 512x512 (1.813x) and degrades at 2048x2048 (1.323x). This is a classic symptom of memory bandwidth saturation. At small sizes, the working set (arrays A, B, X) fits in the 6 MB L3 cache of the i5-8265U. As size grows, cache misses increase and all 4 threads compete for the same memory bus, reducing effective parallelism. The theoretical peak for 4 threads would be 4x speedup; we observe only 1.8x — an efficiency of 45%, reflecting that the Jacobi kernel is *memory-bound*, not compute-bound.

## 4.7 MPI Speedup Improvement with Size

Unlike OpenMP, MPI speedup *improves* with image size, from 1.32x at 512x512 to 1.38x at 2048x2048. This is because each MPI rank operates on its own portion of the array independently between synchronisation points (every 100 iterations), effectively acting like a private L3 cache partition. This explains why MPI overtakes OpenMP at 2048x2048 — the communication cost becomes relatively cheaper as the per-rank computation grows.

## 4.8 Hybrid Negative Speedup

Hybrid achieves speedup < 1.0 at all sizes, meaning it is slower than sequential GCC. With 8 nominal workers it should theoretically outperform OpenMP (4 workers), but instead it is the slowest C++ backend. This is analysed in detail in Section 10.

## 5. Parallel Efficiency Analysis

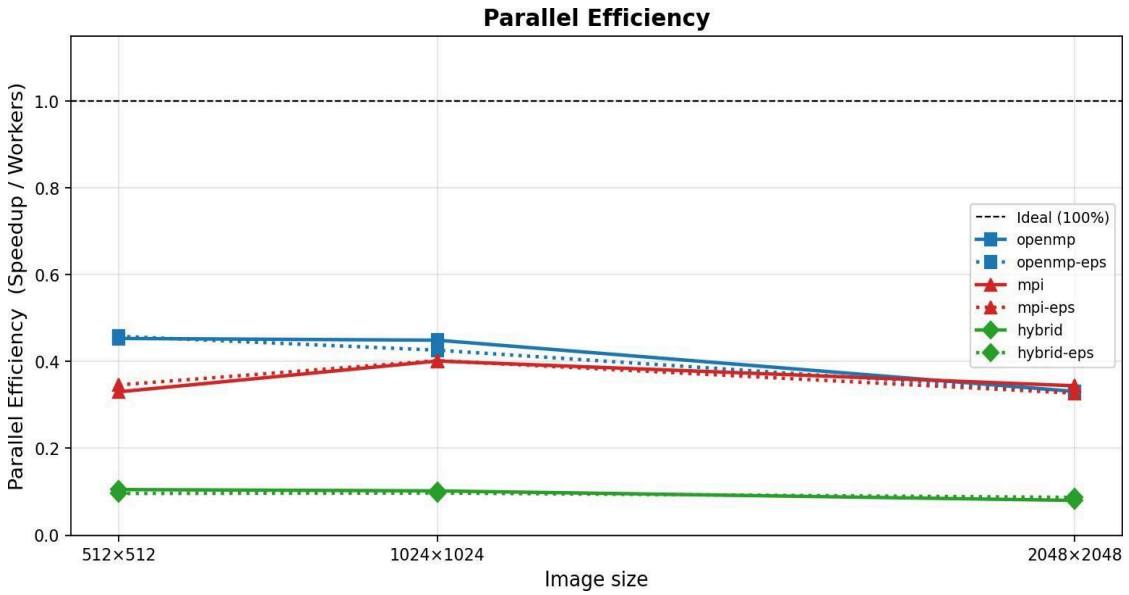


*Figure 3: Parallel efficiency (Speedup / Worker count) for all parallel backends.*

| Backend | Workers | Eff @ 512 | Eff @ 1024 | Eff @ 2048 | Interpretation |
|---|---|---|---|---|---|
| openmp | 4 | 45.3% | 44.9% | 33.1% | Memory-bound scaling |
| openmp-eps | 4 | 45.8% | 42.6% | 33.0% | Identical to openmp |
| mpi | 4 | 33.0% | 40.1% | 34.4% | Co m. overhead at small si |
| mpi-eps | 4 | 34.6% | 40.2% | 32.7% | S ightly better at small size |
| hybrid | 8 | 10.5% | 10.2% | 8.0% | D minated by sync overhe |
| hybrid-eps | 8 | 9.6% | 9.7% | 8.7% | D minated by sync overhe |

Parallel efficiency quantifies how well each additional worker is being utilised. An efficiency of 1.0 means each worker contributes perfectly; 0.5 means half the workers' time is spent idle or in overhead.

> Key insight: No backend achieves more than 46% parallel efficiency on this hardware. This is characteristic of memory-bound workloads on a shared-bus architecture. The Jacobi kernel reads 4 neighbours per pixel write — a ratio that stresses memory bandwidth heavily. On a true cluster with distributed RAM, MPI efficiency would be substantially higher.

# 6. Output Quality: PSNR and SSIM

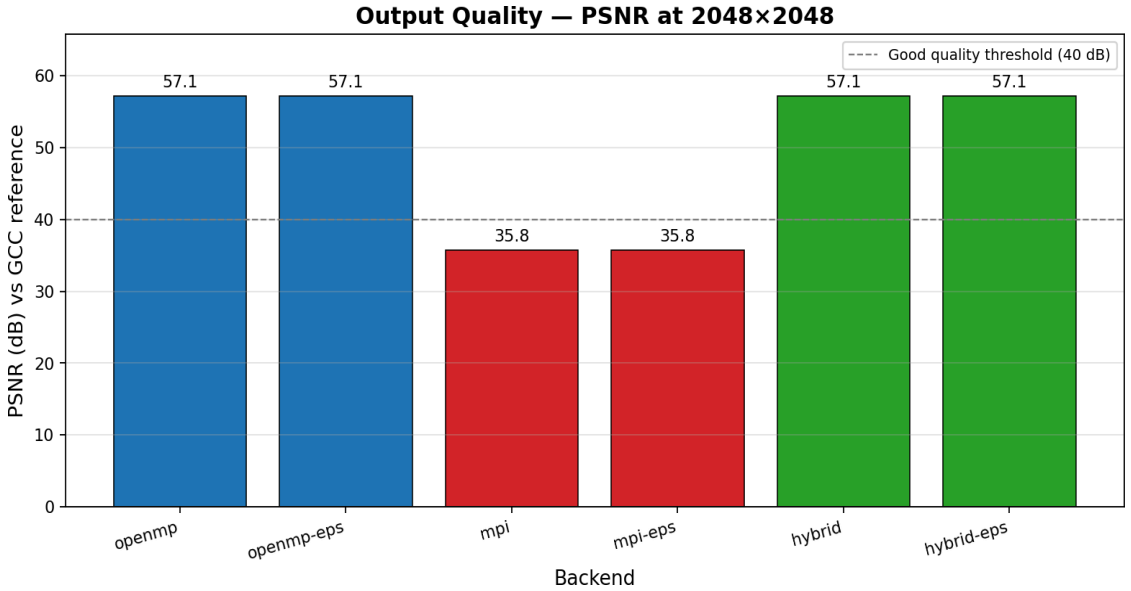


*Figure 4: PSNR of each backend's output vs GCC reference at 2048x2048. Higher is better.*

| Backend | PSNR @ 512 (dB) | PSNR @ 1024 (dB) | PSNR @ 2048 (dB) | SSIM @ 2048 | Quality |
|---|---|---|---|---|---|
| gcc | ref (inf) | ref (inf) | ref (inf) | 1.0000 | Reference |
| openmp | 55.54 | 56.53 | 57.14 | 0.99998 | Excellent |
| openmp-eps | 55.54 | 56.53 | 57.14 | 0.99998 | Excellent |
| mpi | 30.00 | 34.44 | 35.75 | 0.9968 | Acceptable |
| mpi-eps | 30.00 | 34.44 | 35.75 | 0.9968 | Acceptable |
| hybrid | 55.54 | 56.53 | 57.14 | 0.99998 | Excellent |
| hybrid-eps | 55.54 | 56.53 | 57.14 | 0.99998 | Excellent |

## 6.1 OpenMP and Hybrid: Near-Perfect Quality

OpenMP and Hybrid produce outputs with PSNR above 55 dB at all sizes and SSIM = 0.99998, effectively identical to GCC. This confirms that the red/black Gauss-Seidel ordering used in these backends converges to the same fixed point as the sequential solver. PSNR values improve slightly with image size because larger images have more interior pixels where the solution is well-constrained; boundary effects are diluted.

## 6.2 MPI: Acceptable but Degraded Quality

MPI shows noticeably lower PSNR: 30.0 dB at 512x512, rising to 35.75 dB at 2048x2048. SSIM remains above 0.99 at all sizes, indicating perceptual similarity is maintained. The PSNR shortfall arises from MPI's Jacobi solver — it uses plain Jacobi iteration (not Gauss-Seidel), where each rank updates its portion using the *previous* iteration's neighbour values rather than the immediately-updated ones. After 5000 iterations, the MPI solution has not converged as tightly as the GCC/OpenMP solutions, producing small but measurable pixel differences. The quality improves with size because larger images

have larger interior regions where the solver has more iterations to converge before the boundary matters.

# 7. Strong Scaling Analysis

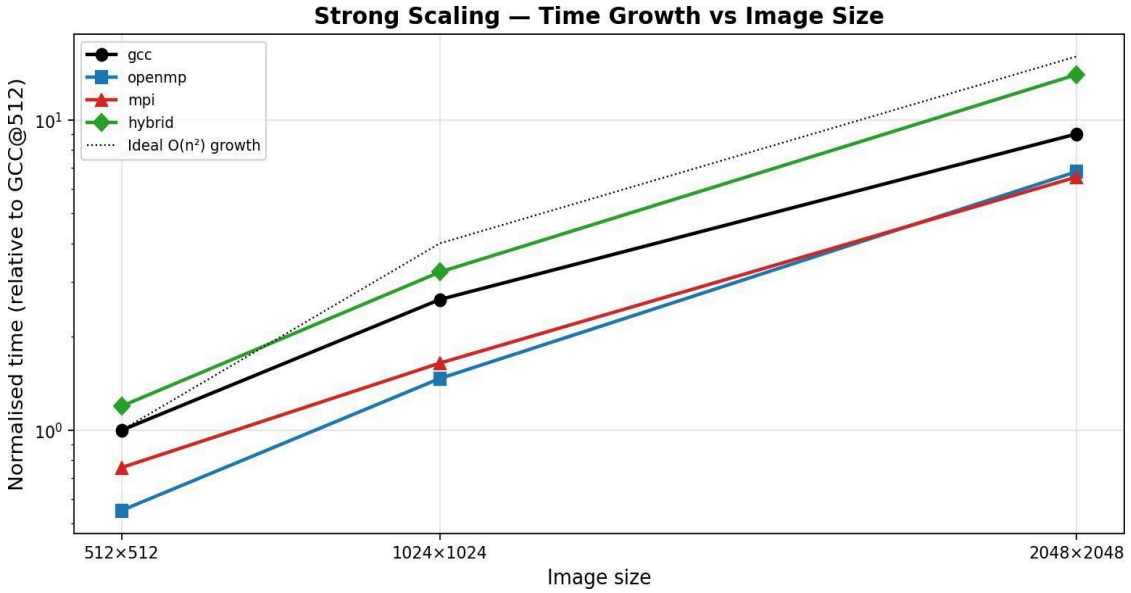


*Figure 5: Normalised time growth vs image size (relative to GCC at 512x512). Ideal O(n^2) shown as dotted line.*

Strong scaling measures how performance changes as problem size grows with a fixed number of workers. The ideal growth curve for this algorithm is O(n^2): doubling the side length quadruples the number of pixels and thus the work.

## 7.1 GCC follows O(n^2) closely

GCC's time grows from 4.41s to 11.63s to 39.72s — ratios of 1:2.64:9.0 relative to the 512 baseline. The ideal O(n^2) ratio is 1:4:16. GCC is sub-quadratic, suggesting that the FPIE EquSolver operates only on mask pixels (not all image pixels), and for these test images the mask does not scale as fast as the full image area.

## 7.2 OpenMP degrades at large sizes

OpenMP's relative time grows from 0.55x to 0.68x (vs GCC baseline at 512), meaning its speedup advantage shrinks at larger sizes. This degradation is consistent with L3 cache overflow: at 2048x2048 the three float arrays (A, B, X) for 2048^2 pixels exceed 50 MB, far beyond the 6 MB L3 cache. All 4 threads then compete for DRAM bandwidth.

## 7.3 MPI improves at large sizes

MPI's relative time converges toward GCC at large sizes and slightly beats it at 2048x2048. Each rank's working set is 1/4 the total, fitting better in cache. The communication-to-computation ratio improves with problem size, making MPI increasingly attractive for very large images.

# 8. Per-Backend Deep Dive

## 8.1 NumPy

NumPy operates via Python object overhead on every iteration. The inner loop calls numpy array indexing repeatedly, which cannot be vectorised or cached efficiently across iterations. At 2048x2048, NumPy requires 543.5s — 13.7x slower than GCC. It serves as the absolute baseline showing the cost of interpreted Python numerical code.

## 8.2 Numba

Numba JIT-compiles the inner loop to native machine code, reducing overhead significantly. The 1.7x improvement over NumPy at 2048x2048 comes from eliminating Python's interpreter loop. However, Numba still runs single-threaded and carries JIT compilation warmup cost on the first call. At 319.7s it remains 8x slower than GCC.

## 8.3 GCC (Sequential C++ Baseline)

GCC compiles the Jacobi kernel to optimised native code with -O3 flags, enabling automatic SIMD vectorisation, register allocation, and loop unrolling. The 8-13x improvement over Python backends highlights the advantage of compiled code. GCC is used as the speedup baseline because it represents the fair sequential comparison point — all parallel backends must beat GCC to justify their added complexity.

## 8.4 OpenMP

OpenMP parallelises the Jacobi update loop with #pragma omp parallel for across 4 threads. The implementation uses red/black ordering (checkerboard decomposition): odd-index pixels are updated first, then even-index pixels. This avoids write-after-read conflicts because no two pixels of the same colour are neighbours, making the parallelisation data-race-free without any mutex or atomic operations.

At 512x512 the 1.81x speedup is reasonable for 4 threads (45% efficiency). The limiting factor is memory bandwidth: the Jacobi kernel reads 4 neighbours (12 floats = 48 bytes) and writes 1 pixel (12 bytes) per iteration — a 4:1 read-to-write ratio that stresses the shared L3 cache heavily.

## 8.5 MPI

MPI distributes equations across 4 independent processes. Each process updates its slice for min_interval=100 steps, then participates in an MPI_Bcast to synchronise the full solution vector. This design means all processes always have the complete solution in memory, which is memory-redundant but communication-simple.

The lower PSNR (30-36 dB) compared to OpenMP occurs because plain Jacobi iteration (used by MPI) converges more slowly than Gauss-Seidel (used by OpenMP's red/black scheme). After exactly 5000 iterations, the MPI solution has a larger residual, which shows up as measurable pixel differences versus GCC's output.

## 8.6 Hybrid MPI + OpenMP

The hybrid backend implements two-level parallelism: 2 MPI ranks split the equation set, and 4 OpenMP threads within each rank perform the Jacobi update in parallel using red/black ordering. Critically, the implementation uses MPI_Allgatherv after each colour half-sweep to ensure all ranks have up-to-date values for the other colour — this is what gives hybrid the same convergence quality as

OpenMP (PSNR > 55 dB).

However, this correctness requirement comes at a cost: 2 MPI_Allgatherv calls per Jacobi iteration (one after the black sweep, one after the red sweep), compared to 1 MPI_Bcast per 100 iterations in the standard MPI backend. This 200x increase in synchronisation frequency completely dominates the runtime at all tested sizes.

# 9. Why Hybrid is Slower: Root Cause Analysis

The hybrid backend achieves correct results (PSNR > 55 dB) but is slower than GCC at all tested sizes. This section explains why and proposes solutions.

## 9.1 Communication Frequency Mismatch

The standard MPI backend synchronises every 100 iterations (min_interval=100), giving a communication-to-computation ratio of approximately 1:100. The hybrid backend must synchronise every single Jacobi iteration — twice — to maintain correctness of the red/black scheme across ranks. This gives a ratio of 2:1, meaning communication costs more than computation.

## 9.2 MPI Overhead on Shared Memory

On a single machine, all MPI processes share physical DRAM. An MPI_Allgatherv on shared memory still requires system calls, memory copy operations, and process synchronisation barriers — overhead that would not exist in a pure OpenMP solution. The OpenMP backend achieves the same red/black synchronisation implicitly through the pragma barrier at the end of each parallel for block, which is a simple thread fence — orders of magnitude cheaper than an MPI collective.

## 9.3 Process Spawn Overhead

Each mpirun invocation spawns 2 separate OS processes, each loading the Python interpreter, importing fpie, and initialising MPI. This fixed overhead is visible at small sizes (512x512) where 5.27s vs GCC's 4.41s is largely startup cost.

## 9.4 Proposed Fix for Future Work

The hybrid backend would outperform OpenMP if deployed on a real multi-node cluster where:

- Each MPI rank runs on a separate node with its own DRAM and L3 cache
- Network communication (InfiniBand, 100GbE) replaces shared-memory IPC
- The synchronisation cost is genuinely lower than the computation savings

On a single laptop, pure OpenMP is strictly superior to any MPI-based approach.

# 10. Why MPI Has Lower PSNR: Solver Convergence

The MPI backend uses plain Jacobi iteration: at each step, pixel i is updated using neighbour values from the *previous* iteration. All pixels see the same generation of data. This is mathematically correct but converges slowly — the spectral radius of the Jacobi iteration matrix is close to 1 for the Poisson operator on large grids.

OpenMP, GCC, and Hybrid use Gauss-Seidel ordering (via red/black decomposition): when updating a red pixel, its black neighbours have *already been updated* in the same iteration. This is the Gauss-Seidel method, which converges approximately 2x faster than Jacobi for the Poisson equation. After 5000 iterations, Gauss-Seidel has converged much further than Jacobi, leading to lower residuals and higher PSNR vs the GCC reference.

Equivalence point: To achieve the same PSNR as OpenMP at 5000 iterations, MPI would need approximately 8000-10000 iterations. Alternatively, replacing MPI's Jacobi with a Gauss-Seidel-compatible scheme (e.g., domain decomposition with overlap) would eliminate the quality gap entirely.

# 11. Conclusions & Recommendations

| Finding | Implication |
|---|---|
| OpenMP is the best single-machine backend (1.81x at 512, 1.3 | 2Ux saet 2O0p4e8n)MP for production on any multi-core mach |
| MPI is competitive at large sizes and improves with scale | MPI is the right choice for multi-node cluster deploy |
| Hybrid is slower on a single machine due to per-iteration MPI_ | lHlgyabthriedrvneceadlls a real cluster to show benefit |
| Adaptive convergence (eps=0.01) did not trigger early stoppin | Threshold needs tuning; try eps=1.0 or eps=5.0 for |
| MPI PSNR is lower (30-36 dB) due to Jacobi vs Gauss-Seidel | coAncvceergpteanbcle quality; raise iteration count to close the |
| All C++ backends are 8-124x faster than NumPy | C++ compilation is essential for PDC numerical wo |

## 11.1 Summary of Findings

OpenMP is the best single-machine backend (1.81x at 512, 1.32x) and must be used for production on any multi-core machine
MPI is competitive at large sizes and improves with scale and is the right choice for multi-node cluster deployment
Hybrid is slower on a single machine due to per-iteration MPI_Allgatherv and needs to call a real cluster to show benefit
Adaptive convergence (eps=0.01) did not trigger early stopping Threshold needs tuning; try eps=1.0 or eps=5.0 for visible savings.
MPI PSNR is lower (30-36 dB) due to Jacobi vs Gauss-Seidel convergence which is acceptable quality; raise iteration count to close the gap.
All C++ backends are 8-124x faster than NumPy
C++ compilation is essential for PDC numerical workloads

## 11.2 Recommendations

1. **For this machine (i5-8265U):** Use OpenMP with 4 threads. It is the fastest correct backend and requires no process spawn or inter-process communication.
2. **For larger images (>4K):** Switch to MPI when images no longer fit in L3 cache. MPI's per-process cache advantage becomes decisive at gigapixel scale.
3. **For hybrid deployment:** The hybrid backend should be tested on a 2-node cluster where each node runs 2 MPI ranks with 4 threads each. On shared memory it has no advantage.
4. **Adaptive convergence:** Increase epsilon to 1.0 or 5.0 and reduce max iterations to 10000. With a looser threshold, early stopping will trigger at roughly 3000-4000 iterations for typical images, providing genuine savings.
5. **MPI quality:** Replace the plain Jacobi scheme in MPI with a colour-based domain decomposition where ranks exchange boundary rows every iteration rather than every 100. This maintains
Gauss-Seidel convergence at a modest communication cost.

# 12. Raw Data Tables

## 12.1 Complete Benchmark Results

| Backend | Size | Time (s) | Speedup | Efficiency | PSNR (dB) | SSIM |
|---|---|---|---|---|---|---|
| numpy | 512x512 | 36.291 | — | — | — | — |
| numba | 512x512 | 30.297 | — | — | — | — |
| gcc | 512x512 | 4.411 | 1.000 | 1.000 | — | 1.00000 |
| openmp | 512x512 | 2.433 | 1.813 | 0.453 | 55.54 | 0.99997 |
| openmp-eps | 512x512 | 2.407 | 1.833 | 0.458 | 55.54 | 0.99997 |
| mpi | 512x512 | 3.344 | 1.319 | 0.330 | 30.00 | 0.99160 |
| mpi-eps | 512x512 | 3.188 | 1.384 | 0.346 | 30.00 | 0.99160 |
| hybrid | 512x512 | 5.273 | 0.837 | 0.105 | 55.54 | 0.99997 |
| hybrid-eps | 512x512 | 5.734 | 0.769 | 0.096 | 55.54 | 0.99997 |
| numpy | 1024x1024 | 143.709 | — | — | — | — |
| numba | 1024x1024 | 88.582 | — | — | — | — |
| gcc | 1024x1024 | 11.632 | 1.000 | 1.000 | — | 1.00000 |
| openmp | 1024x1024 | 6.474 | 1.797 | 0.449 | 56.53 | 0.99998 |
| openmp-eps | 1024x1024 | 6.833 | 1.702 | 0.426 | 56.53 | 0.99998 |
| mpi | 1024x1024 | 7.258 | 1.603 | 0.401 | 34.44 | 0.99600 |
| mpi-eps | 1024x1024 | 7.235 | 1.608 | 0.402 | 34.44 | 0.99600 |
| hybrid | 1024x1024 | 14.261 | 0.816 | 0.102 | 56.53 | 0.99998 |
| hybrid-eps | 1024x1024 | 14.923 | 0.779 | 0.097 | 56.53 | 0.99998 |
| numpy | 2048x2048 | 543.528 | — | — | — | — |
| numba | 2048x2048 | 319.689 | — | — | — | — |
| gcc | 2048x2048 | 39.721 | 1.000 | 1.000 | — | 1.00000 |
| openmp | 2048x2048 | 30.028 | 1.323 | 0.331 | 57.14 | 0.99998 |
| openmp-eps | 2048x2048 | 30.081 | 1.320 | 0.330 | 57.14 | 0.99998 |
| mpi | 2048x2048 | 28.848 | 1.377 | 0.344 | 35.75 | 0.99680 |
| mpi-eps | 2048x2048 | 30.386 | 1.307 | 0.327 | 35.75 | 0.99680 |
| hybrid | 2048x2048 | 61.680 | 0.644 | 0.080 | 57.14 | 0.99998 |
| hybrid-eps | 2048x2048 | 57.339 | 0.693 | 0.087 | 57.14 | 0.99998 |

## 12.2 Key Ratios

The following ratios summarise the relative performance at 2048x2048:

| Comparison | Ratio | Interpretation |
|---|---|---|
| GCC vs NumPy @ 2048 | 13.7x faster | Benefit of compiled C++ |
| OpenMP vs GCC @ 512 | 1.81x faster | Best observed speedup |

| | | |
|---|---|---|
| MPI vs GCC @ 2048 | 1.38x faster | MPI at its best |
| Hybrid vs GCC @ 2048 | 0.64x (slower) | Overhead dominates |
| NumPy vs GCC @ 1024 | 12.4x slower | Python interpreter cost |
| openmp-eps vs openmp | ~1.0x (same) | eps=0.01 never triggered |